



UNIVERSITY OF GLOBAL VILLAGE (UGV), BARISHAL
Department of Computer Science and Engineering

Object-Oriented Programming with Java

Course Code: CSE 0613-2103



Let's Think About This

How do you organize a messy room?

- You group books together, clothes together, tools together.
- Each group has things that belong together — data and purpose.
- That's exactly what OOP does with code.



Procedural vs. OOP

Procedural: a list of steps, like a recipe.

OOP: a team of objects that talk to each other.

What Is OOP?

Object-Oriented Programming is a programming paradigm that models software as a collection of interacting objects, rather than a sequence of instructions.

Core idea

- Every object bundles DATA (attributes/state) with BEHAVIOR (methods/actions).
- Objects are created from CLASSES — reusable blueprints.
- Objects communicate by calling each other's methods.
- Code becomes modular: change one object without breaking others.

Real-Life Example(Student)

Attributes:

- Name
- ID
- Department
- Semester

Methods:

- Study()
- AttendClass()
- SubmitAssignment()

- Every student is an object.

Class vs. Object

A class is the blueprint. An object is a real instance built from it — and you can build many objects from one class.



CLASS

The blueprint

House blueprint — defines rooms, doors, windows, but you can't live inside a blueprint.

```
class House { ... }
```



OBJECT

The real instance

Your actual house — and your neighbor's house is a different object from the SAME class, with its own values.

```
House h1 = new House();  
House h2 = new House();
```

A Simple Java Example

A Student class (the blueprint) and a Main class that creates and uses a Student object.

```
class Student {  
    String name;  
    int id;  
  
    void printInfo() {  
        System.out.println(name + " (ID: " + id + ")");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "Rahim";  
        s1.id = 101;  
        s1.printInfo();  
    }  
}
```



How to read it

- Student is the **CLASS** — name and id are its attributes.
- printInfo() is a **METHOD** — behavior the object can perform.
- new Student() creates an **OBJECT** called s1.
- s1.name and s1.id set that object's own data.
- s1.printInfo() runs the method using s1's data.

What Happens When We Run It

Java runs `main()`, builds one `Student` object in memory, then prints using its data.



```
> java Main  
Rahim (ID: 101)
```



Object s1 in memory

```
name  
"Rahim"  
  
id  
101
```



Step by step

- `main()` starts running — this is the entry point of every Java program.
- `new Student()` reserves memory for a fresh object and returns its reference into `s1`.
- `s1.name = "Rahim"` and `s1.id = 101` fill in that object's own attributes.
- `s1.printInfo()` looks up name and id ON `s1` and prints them.
- If we made a second student `s2`, it would have its own separate name and id.

The Four Pillars of OOP

Every OOP language — including Java — is built on these four ideas. We'll look at each one with code next.



Encapsulation

hide the wiring



Abstraction

show what matters



Inheritance

*share common
traits*



Polymorphism

*one action, many
forms*

Encapsulation

Hide internal details; expose only what's needed through controlled access.



Example: an ATM

You insert a card and get cash. You never see the wiring, the bank's database, or the internal logic — just a simple interface.

- private hides a field from outside code.
- public methods (getters/setters) control access.
- Protects data from accidental or invalid changes.

```
class Account {  
    private double balance;  
  
    public void deposit(double amt) {  
        if (amt > 0) balance += amt;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}  
  
// Outside code can't touch  
// 'balance' directly -- only  
// through deposit() / getBalance()
```


Abstraction

Show only relevant features; hide unnecessary complexity behind a simple interface.



Example: driving a car

You use a steering wheel and pedals. You don't need to understand the engine, fuel injection, or transmission to drive.

- abstract class defines WHAT to do, not HOW.
- Subclasses fill in the implementation details.
- Callers only interact with the simple, shared method.

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    double radius;  
  
    double area() {  
        return 3.14 * radius * radius;  
    }  
}  
  
// Caller just calls shape.area() --  
// no need to know the formula used
```

Inheritance

A new class reuses and extends the fields and methods of an existing one.



Example: Vehicle family

"Vehicle" is a parent with an engine and wheels. "Car", "Bike", and "Truck" inherit those traits but each adds its own extras.

- extends lets a subclass reuse a parent's code.
- Avoids rewriting the same fields/methods repeatedly.
- Subclasses can still add their own new behavior.

```
class Vehicle {  
    void start() {  
        System.out.println("Starting...");  
    }  
}  
  
class Car extends Vehicle {  
    void honk() {  
        System.out.println("Beep!");  
    }  
}  
  
// Car car = new Car();  
// car.start(); <- inherited  
// car.honk(); <- Car's own
```

Polymorphism

The same method call behaves differently depending on the actual object.



Example: the Play button

Pressing "Play" starts a song on a music app but starts a video on a video app — same button, same word, different result.

- A subclass can override a parent's method.
- The correct version runs automatically at call time.
- One method name, many object-specific behaviors.

```
class Animal {  
    void sound() {  
        System.out.println("Some sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}  
  
// Animal a = new Dog();  
// a.sound(); -> prints "Bark"
```

Why OOP Matters in the Real World

This isn't just theory — OOP is how most real software is built and maintained.



Mobile apps

Every screen, button, and account in your favorite app is an object.



Banking systems

Accounts, transactions, and cards are modeled as interacting objects.



Video games

Characters, weapons, and enemies are all objects with their own behavior.



Team collaboration

Different developers build different classes that fit together like puzzle pieces.

Today's Task

Development Environment Setup & Basic Java Programming.



Part 1 — Install Apache NetBeans IDE

Install the latest version of **Apache NetBeans IDE** on your personal computer.

Requirements:

- Install the **Java Development Kit (JDK 21 or later)**.
- Install **Apache NetBeans IDE**.
- Verify the installation by creating and running a simple Java project.



Part 2 - Write Your First Java Program

Create a Java program that displays the following information on the console:

University of Global Village
Department of Computer Science & Engineering

Name : Your Name
Student ID: Your ID
Semester : Your Semester

Welcome to Object-Oriented Programming with Java.

Simple Question's



What is OOP? Why do we need OOP instead of Procedural Programming?



What is Class and Object in OOP? Difference between a class and an object?



Identify three objects you use every day and mention one attribute of each.



What is the difference between an attribute and a method? Give one example.



Questions?